

TP9 — Playbook avec gestion complète des exceptions

Formation NetOps : Automatiser la gestion de son Infrastructure Réseau — Ambient IT

Description

Ce TP ajoute une logique avancée à la vérification des interfaces (TP5-TP8) : des tentatives automatiques en cas d'échec transitoire (`retries/until`), une gestion structurée des erreurs avec `block/rescue/always` (pour qu'une panne sur un site n'interrompe pas la vérification des autres), une condition complexe combinant plusieurs variables, et une boucle qui accumule un bilan des échecs par site.

Prérequis

- TP1 à TP8 validés

Étapes

Étape 1 — Créer sa branche

```
cd netops-project-<nom-groupe>
git checkout dev
git pull origin dev
git checkout -b <prenom>-<nom>-jour3-tp9
```

Étape 2 — Créer le fichier de tâches avec gestion des exceptions

```
cat > roles/base_config/tasks/check_interface.yml << 'EOF'
---
- name: "Verifier l'interface {{ item.name }} (gestion des exceptions)"
  block:
    - name: "Interroger l'API pour {{ item.name }}"
      ansible.builtin.uri:
        url: "{{ api_url }}/api/devices/{{ device_id_prefix }}-{{ inventory_hostname }}-01/interfaces"
        method: GET
        headers:
          X-Api-Token: "{{ vault_api_token }}"
        return_content: yes
      register: iface_result
      until: iface_result.status == 200
```

```

    retries: 3
    delay: 2

- name: "Extraire l'etat reel de {{ item.name }}"
  ansible.builtin.set_fact:
    etat_reel: "{{ iface_result.json.interfaces[item.name].status }}"

- name: "Alerter si incoherence entre config souhaitee et etat reel"
  ansible.builtin.debug:
    msg: "INCOHERENCE sur {{ inventory_hostname }}/{{ item.name }} : configuree {{ item.enabled }} mais etat reel {{ etat_reel }}"
  when: >
    (item.enabled == 'up' and etat_reel == 'down') or
    (item.enabled == 'down' and etat_reel == 'up')

rescue:
- name: "Consigner l'echec de verification pour {{ item.name }}"
  ansible.builtin.set_fact:
    interfaces_en_echec: "{{ (interfaces_en_echec | default([])) + [item.name] }}"

- name: "Afficher l'erreur rencontree, sans arreter le reste du playbook"
  ansible.builtin.debug:
    msg: "ECHEC verification de {{ item.name }} sur {{ inventory_hostname }} : {{ ansible_failed_result.msg | default('erreur inconnue') }}"

always:
- name: "Marquer l'interface comme traitee"
  ansible.builtin.debug:
    msg: "Interface {{ item.name }} traitee sur {{ inventory_hostname }}"
    verbosity: 1
EOF

```

Étape 3 — Mettre à jour le rôle pour boucler sur les interfaces

```

cat > roles/base_config/tasks/main.yml << 'EOF'
---
- name: Verifier que le site repond (health check)
  ansible.builtin.uri:
    url: "{{ api_url }}/health"
    method: GET
    return_content: yes
    register: health_result

- name: Afficher le statut du site
  ansible.builtin.debug:
    msg: "Site {{ health_result.json.site }} : {{ health_result.json.status }}"

- name: Verifier chaque interface avec gestion complete des exceptions
  ansible.builtin.include_tasks: check_interface.yml
  loop: "{{ interfaces }}"

```

```
loop_control:
  label: "{{ item.name }}"
```

```
- name: Afficher le bilan des interfaces en echec sur ce site
  ansible.builtin.debug:
```

```
msg: "Bilan {{ inventory_hostname }} : {{ interfaces_en_echec | default([]) | length }} interface(s) en echec -> {{ interfaces_en_echec | default([]) }}"
```

E0F

Étape 4 — Tester l'exécution normale

```
cd lab && docker compose up -d --build && cd ..
ansible-playbook --vault-password-file .vault_pass playbooks/site.yml
```

Résultat attendu : tous les sites en ok, Bilan <site> : 0 interface(s) en echec pour chacun.

Étape 5 — Tester la condition complexe (incohérence détectée)

```
sed -i "s/enabled: down/enabled: up/" inventory/host_vars/siege.yml
ansible-playbook --vault-password-file .vault_pass playbooks/site.yml --limit siege
```

Résultat attendu : le message INCOHERENCE sur siege/GigabitEthernet0/2 : configuree up mais etat reel down apparaît — la condition combine deux informations (l'état souhaité venu de host_vars, l'état réel venu de l'API) pour détecter un écart.

Remettez la valeur d'origine ensuite (rééditez le fichier pour repasser GigabitEthernet0/2 à enabled: down).

Étape 6 — Tester le rescue (une vraie panne, pas juste une incohérence)

6.1 — Vérifier l'état actuel avant de casser quoi que ce soit

```
curl -s http://localhost:5001/health
```

Doit répondre {"site":"siege","status":"ok"}.

6.2 — Casser volontairement le token de siege-router uniquement

```
sed -i '/siege-router:\/,\/networks:\/ s/API_TOKEN=${API_TOKEN}/API_TOKEN=TOKEN_CASSE/' lab/docker-compose.yml
```

Vérifiez que seul siege-router a changé :

```
grep -A 3 "container_name" lab/docker-compose.yml
```

6.3 — Recréer uniquement le conteneur siege-router

```
cd lab
docker compose up -d --build siege-router
cd ..
```

6.4 — Vérifier que la panne est bien en place

```
curl -s -H "X-API-Token: s3cr3t-token-nexacorp" http://localhost:5001/api/devices/router-siege-01/interfaces
```

Doit répondre {"error": "Token invalide ou manquant"} (401).

6.5 — Lancer le playbook complet

```
ansible-playbook --vault-password-file .vault_pass playbooks/site.yml
```

Ce que vous devez observer, dans l'ordre :

1. siege : la tâche Interroger l'API pour GigabitEthernet0/1 échoue, avec des lignes FAILED - RETRYING (jusqu'à 3 tentatives, 2 secondes d'intervalle)
2. Après épuisement des tentatives, le rescue prend le relais : Consigner l'echec... puis Afficher l'erreur rencontrée...
3. Le message ECHEC verification de GigabitEthernet0/1 sur siege : Status code was 401... s'affiche
4. Le playbook **continue** : GigabitEthernet0/2 sur siege subit le même sort, puis agence1 et agence2 s'exécutent normalement
5. Bilan siege : 2 interface(s) en echec -> [...], alors que Bilan agence1 et Bilan agence2 affichent 0

6.6 — Vérifier le récapitulatif final

```
agence1      : ok=9    changed=0    ... failed=0    rescued=0
agence2      : ok=9    changed=0    ... failed=0    rescued=0
siege        : ok=9    changed=0    ... failed=0    rescued=2
```

rescued=2 sur siege confirme que les 2 échecs ont bien été interceptés et gérés — failed=0 partout signifie qu'aucun hôte n'a fait échouer le playbook dans son ensemble.

6.7 — Réparer et revenir à l'état normal

```
sed -i 's/API_TOKEN=TOKEN_CASSE/API_TOKEN=${API_TOKEN}/' lab/docker-compose.yml
cd lab
docker compose up -d --build siege-router
cd ..
ansible-playbook --vault-password-file .vault_pass playbooks/site.yml
```

Doit repasser avec Bilan siege : 0 interface(s) en echec.

Étape 7 — Committer et pousser

```
git add roles/base_config/
git commit -m "TP9 : gestion complete des exceptions (block/rescue/always, retries, conditions complexes)"
git push -u origin <prenom>-<nom>-jour3-tp9
```

Ouvrir une pull request <prenom>-<nom>-jour3-tp9 → dev.

Comprendre le TP

until / retries / delay : Ansible réexécute la tâche jusqu'à ce que la condition until soit vraie, ou jusqu'à épuisement des tentatives (retries), en attendant delay secondes entre chaque essai. Utile pour les pannes transitoires — sans ça, une seule coupure d'une seconde ferait échouer tout le déploiement.

block / rescue / always : l'équivalent d'un try/except/finally. Le block contient le travail normal ; si une tâche à l'intérieur échoue, rescue prend le relais au lieu de stopper tout le playbook ; always s'exécute dans tous les cas, succès ou échec. C'est ce qui permet à une panne sur siege de ne pas empêcher la vérification d'agence1 et agence2.

Ce que rescued signifie concrètement : à chaque fois qu'une tâche du block échoue, Ansible bascule directement dans le rescue au lieu d'arrêter le hôte. Le compteur rescued du récapitulatif final compte combien de fois ce basculement a eu lieu pour cet hôte. C'est différent de failed (une tâche qui échoue et qu'aucun rescue ne rattrape) et de ignored (ignore_errors: yes, qui ignore l'échec sans logique de traitement particulière). Sans rescue, la même panne aurait donné failed=1 sur siege et arrêté immédiatement le traitement de ce site (plus de vérification de la deuxième interface, pas de Bilan affiché) — mais aurait quand même laissé agence1/agence2 continuer, un échec sur un hôte n'affectant pas les autres hôtes par défaut. Le rescue va plus loin : il permet à siege lui-même de terminer normalement malgré la panne sur sa première interface.

Condition complexe (when combinant deux critères) : la ligne (item.enabled == 'up' and etat_reel == 'down') or (item.enabled == 'down' and etat_reel == 'up') compare deux sources différentes (la configuration souhaitée du TP6, l'état réel renvoyé par l'API) pour détecter un écart.

Accumulateur dans une boucle (interfaces_en_echec | default([]) + [item.name]) : à chaque itération en échec, on ajoute le nom de l'interface à une liste qui grossit progressivement — le pattern standard Ansible pour collecter des résultats au fil d'une boucle.

Validation

- ☐ Exécution normale : tous les sites en ok, 0 interface en échec
- ☐ Une incohérence volontaire (enabled modifié) déclenche le message INCOHERENCE
- ☐ Une vraie panne sur un site (token cassé) déclenche le rescue, s'affiche clairement, et n'arrête pas la vérification des autres sites
- ☐ Le récapitulatif final montre rescued=2 sur le site en panne, 0 ailleurs
- ☐ Bilan <site> reflète correctement le nombre d'interfaces en échec par site
- ☐ Une pull request <prenom>-<nom>- jour3-tp9 → dev est ouverte